Computing Platform for Autonomous Driving (CP4AD)

3. RTOS #2

Prof. Jong-Chan Kim

Dept. Automobile and IT Convergence

KMU 국민대학교 KOOKMIN UNIVERSITY

Counter

- Abstraction of hardware clock tick
- Three relavant parameters
  - TicksPerBase
  - MaxAllowedValue
  - MinCycle

```
ISR(timer_handler)
{
    IncrementCounter(myCounter);
}
```

increment the counter

```
COUNTER myCounter {
    TICKSPERBASE = 1;
    MAXALLOWEDVALUE = 65535;
    MINCYCLE = 1;
}
```

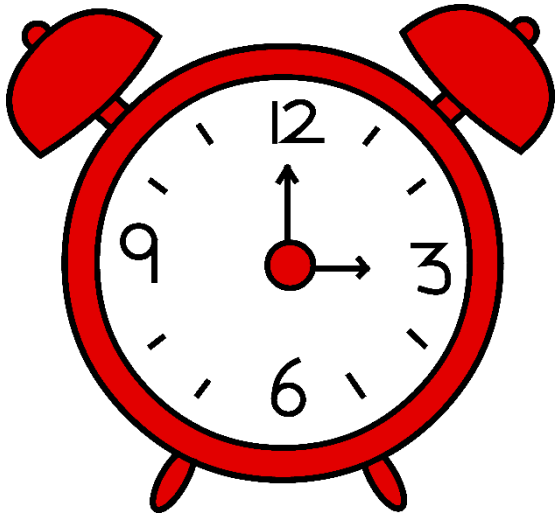
number of interrupts that increases the counter value by one

min interval bet'n two triggering of the alarm

maximum number of counter (reset to 0 after reaching this value)

Alarm

- Alarm is connected to a counter to perform an action
- When a counter reaches a value of the alarm, the pre-defined actions are performed
  - Set an event
  - Activate a task
  - Function call



OIL Description of Alarm

```
ALARM myAlarm {
    COUNTER = myCounter;
    ACTION = ACTIVATETASK {
        TASK = myTask;
    }
    AUTOSTART = TRUE {
        ALARMTIME = 10;
        CYCLETIME = 5000;
    }
}
```

first triggered at 10

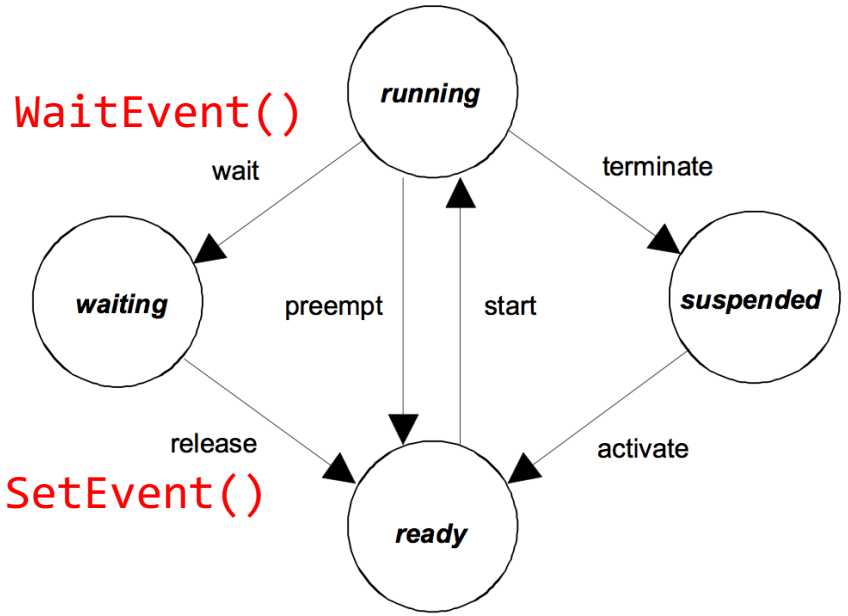
periodically triggered at every 5000 counter ticks

Event

- WaitEvent() makes a task voluntarily release CPU (running ➔ waiting)
- SetEvent() makes the waiting task get back (waiting ➔ ready)

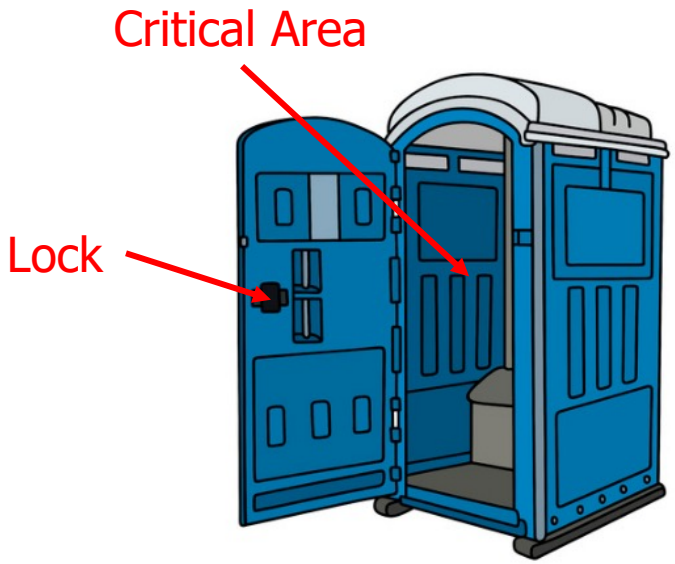
```
TASK(Task1)
{
    ...
    WaitEvent(event);
    ...
    TerminateTask();
}
```

```
TASK(Task2)
{
    ...
    SetEvent(Task1, event);
    ...
    TerminateTask();
}
```



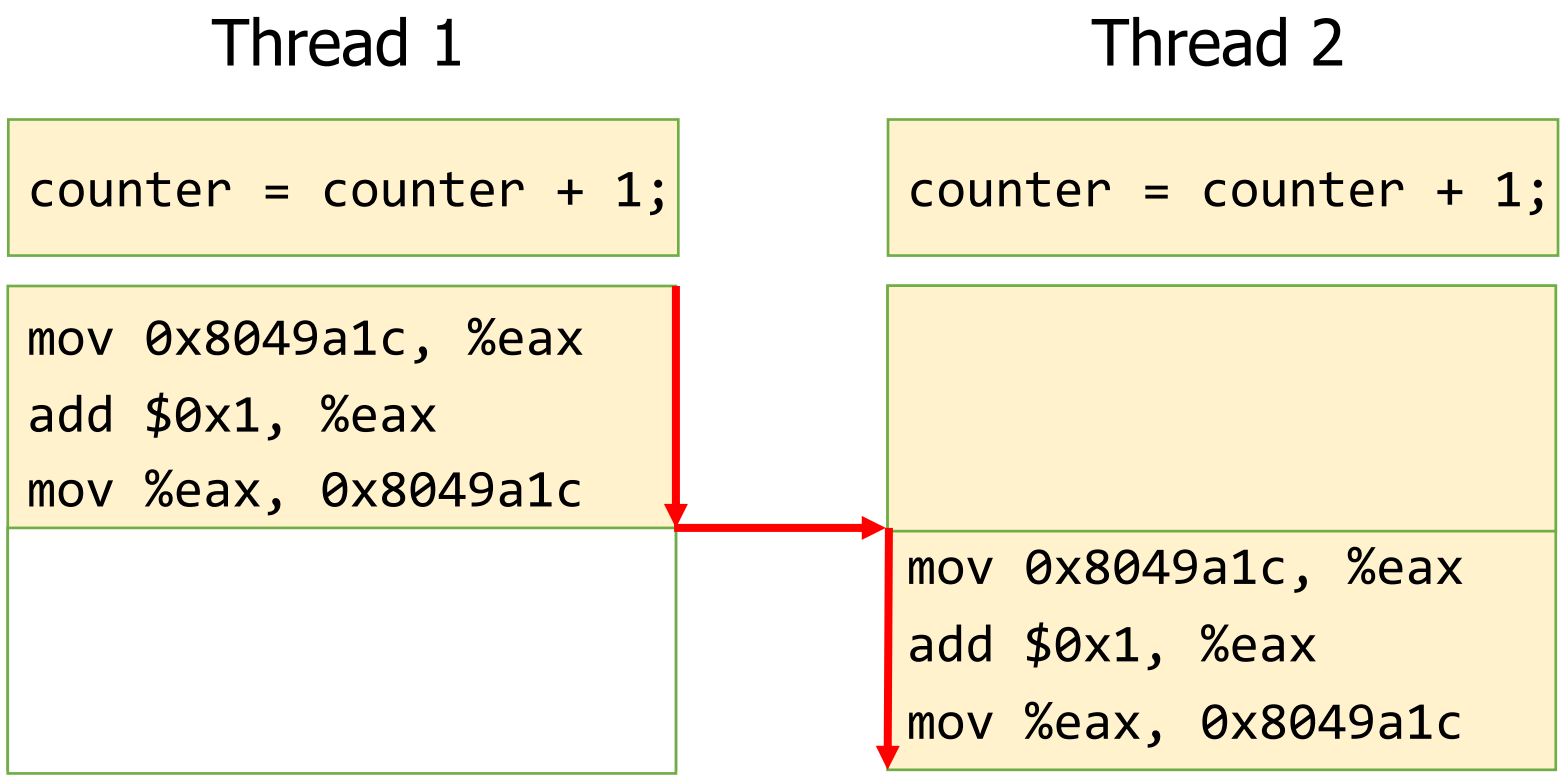
Shared Resource

- Multiple users (i.e., thread) use the same resource (e.g., toilet booth)
- The resource should be used in isolation (mutual exclusion)
- Users agree on entering the area by checking and locking the lock
- In computer systems, there are many shared resources
  - Global or static variables
  - Files
  - Shared memory areas
  - Hardware registers
  - …
- Cause of many **race conditions**



Race Condition (1/2)

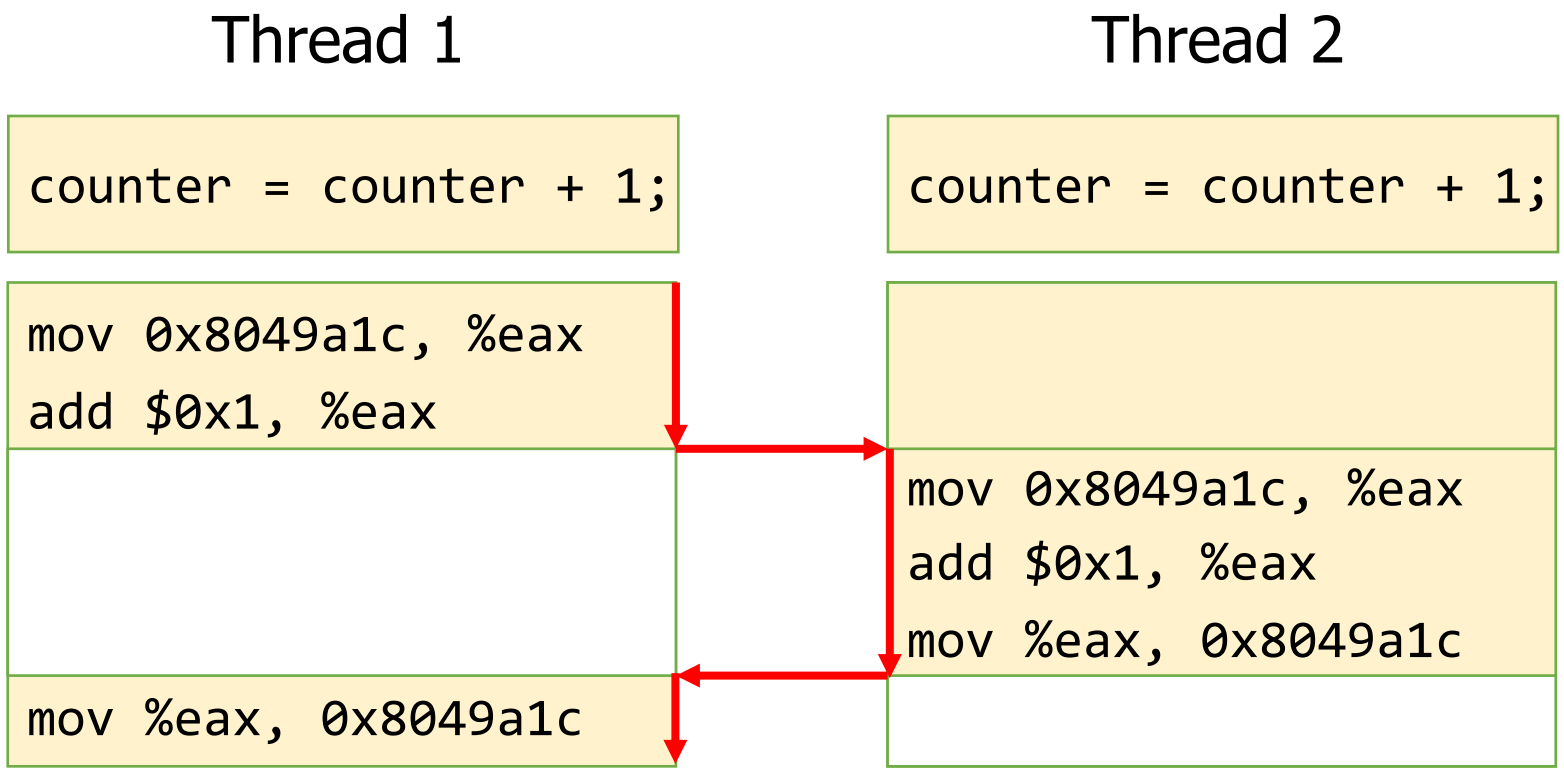
- When result depends on sequence or timing of uncontrollable events
  - For example, scheduling events (i.e., preemptions)



Works as intended

Race Condition (2/2)

- What if a preemption happens as following?



Functionally incorrect (even unpredictable) result

Shared Resource Integrity Problem

```
unsigned int loop_cnt;
volatile unsigned int counter = 0;
void *mythread(void *arg)
{
    for (i = 0; i < loop_cnt; i++)
        counter = counter + 1;
    return NULL;
}
int main(int argc, char *argv[])
{
    loop_cnt = atoi(argv[1]);

    pthread_t p1, p2;
    pthread_create(&p1, NULL, mythread, "A");
    pthread_create(&p2, NULL, mythread, "B");
    // join waits for the threads to finish
    pthread_join(p1, NULL);
    pthread_join(p2, NULL);
    printf("%d\n", counter);
    return 0;
}
```

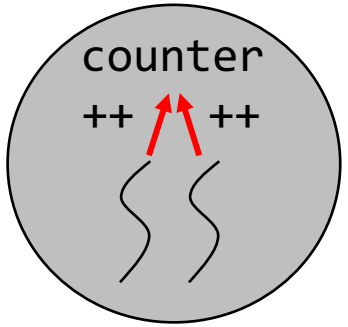
Shared global variable by p1 and p2

counter variable's address

```
…
mov 0x8049a1c, %eax
add $0x1, %eax
mov %eax, 0x8049a1c
…
```

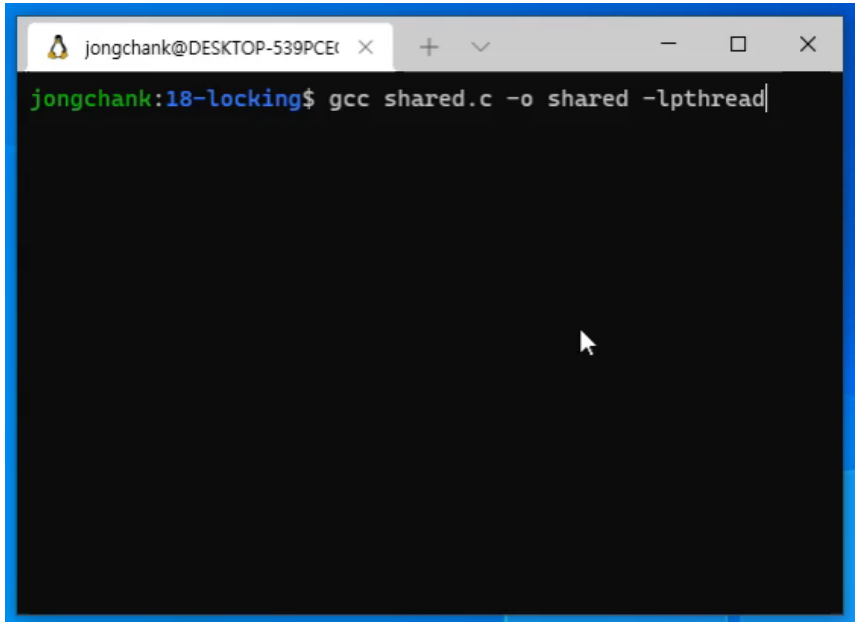
CPU register

Code will not compile since details have been removed for ease of explanation.




```
$ gcc shared.c -o shared –lpthread
$ ./shared 1000000
```

What do you expect?

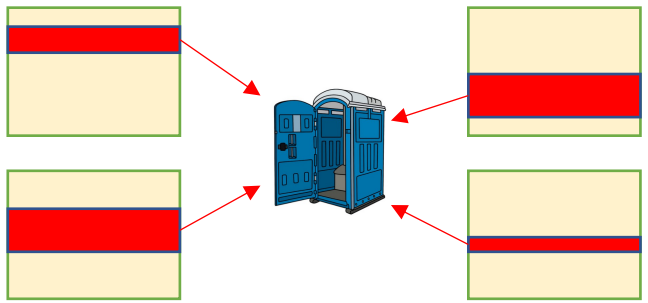


Mutual Exclusion for Critical Section

- Critical Section
  - Code segment for handling a shared resource
    - Single shared resource can make many critical sections in different threads sharing the resource
- Mutual exclusion
  - Only a single thread in the critical section
- Atomicity
  - counter = counter + 1; is not atomic
    - Three instructions

```
void *mythread(void *arg)
{
    for (i = 0; i < max; i++)
        counter = counter + 1;
        mov 0x8049a1c, %eax
        add $0x1, %eax
        mov %eax, 0x8049a1c
    return NULL;
}
```

Critical Section



How to protect critical section

- Lock (also called Mutex)
  - Mutual agreements between threads
  - Only one thread holding the lock enters
- Lock Variable
  - A shared variable
    - Lock state (Locked or not)
    - Lock owner (Who is holding the lock)

A mutex can be unlocked by who locked it

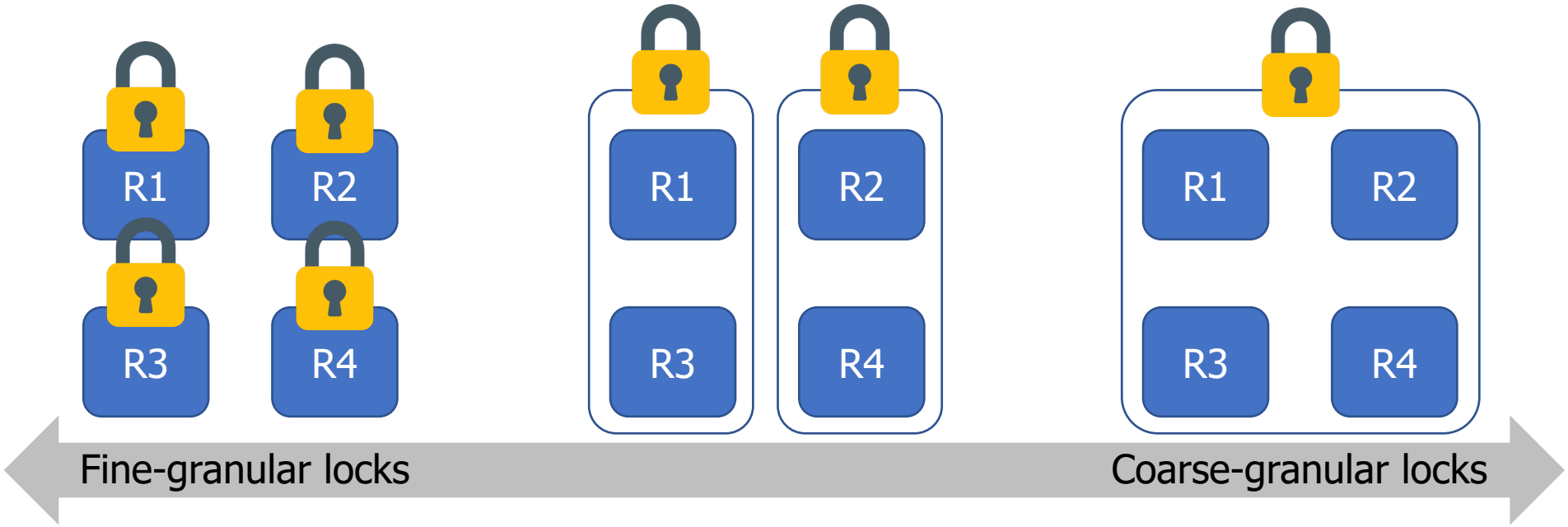
```
lock_t mutex;
void *mythread(void *arg)
{
    for (i = 0; i < max; i++)
{
        lock(&mutex);
        counter = counter + 1;
        unlock(&mutex);
    }
    return NULL;
}
```

What if lock() fails to get the lock?

- Spinlock
  - Loops continually checking the lock variable
  - Wastes CPU time
  - No additional delay before entering the critical section
- Mutex (Normal blocking lock)
  - Blocks the calling thread and puts it to waiting queue
  - Does not waste CPU time
  - Unlock() wakes up the blocking threads
  - Delays for
    - Moving threads from waiting queue to ready queue
    - Selecting a new thread

Lock Granularity

- Fine-granular method
  - Individual lock for each shared resource
  - Lots of lock variables with increased code complexity
- Coarse-granular method
  - Small number of locks after grouping shared resources
  - Unnecessary blockings by shared lock variables (inefficiency)

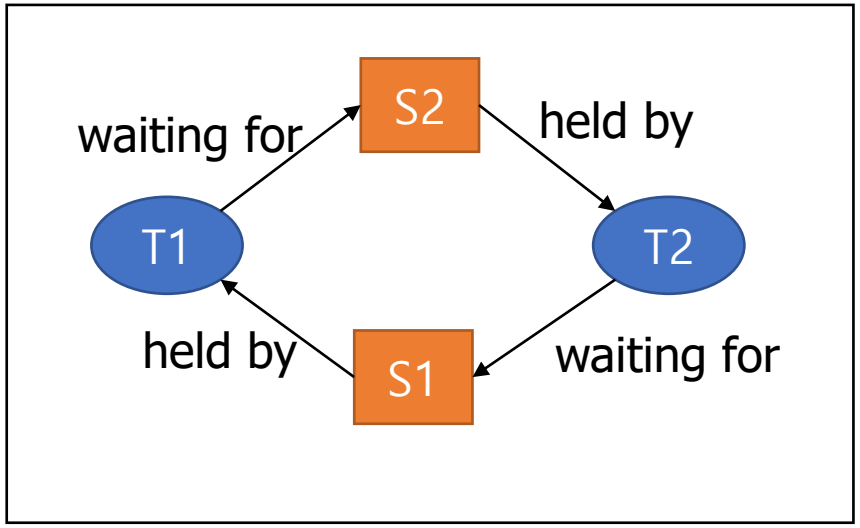


Problems Caused by Locks

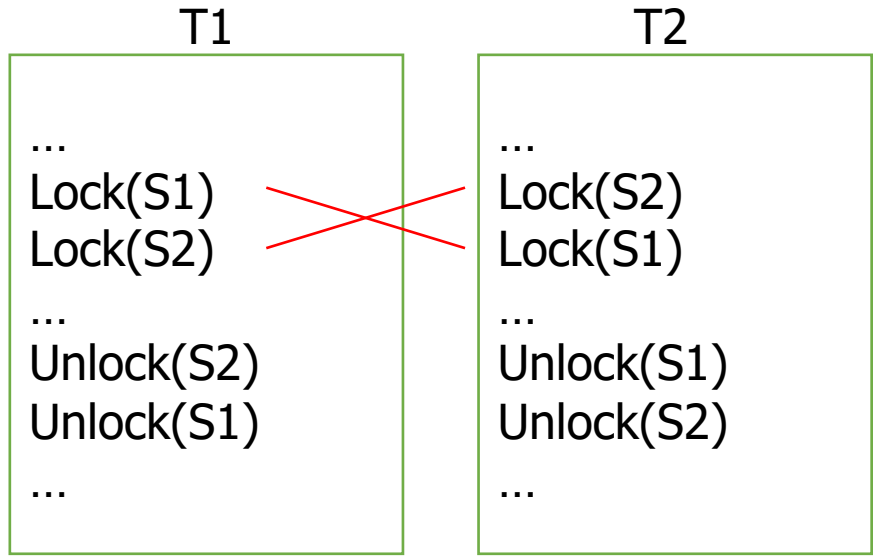
- Deadlock
  - Two or more tasks are waiting for each other to unlock first
- Priority Inversion
  - A higher priority task is blocked by lower priority tasks

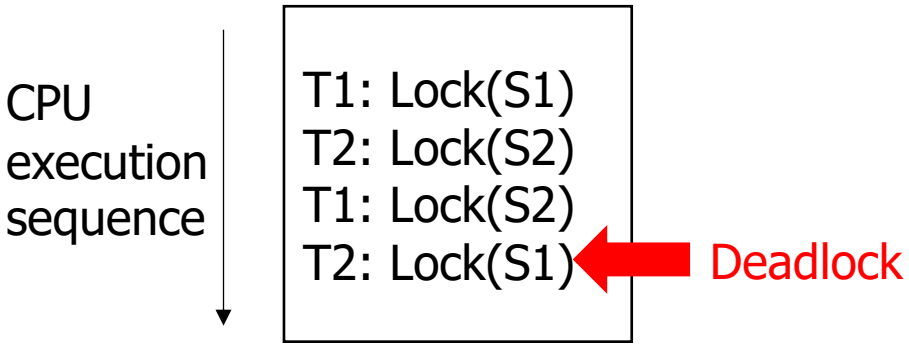
Deadlock

- Happens when two tasks try to acquire multiple nested locks in different orders



Circular wait example



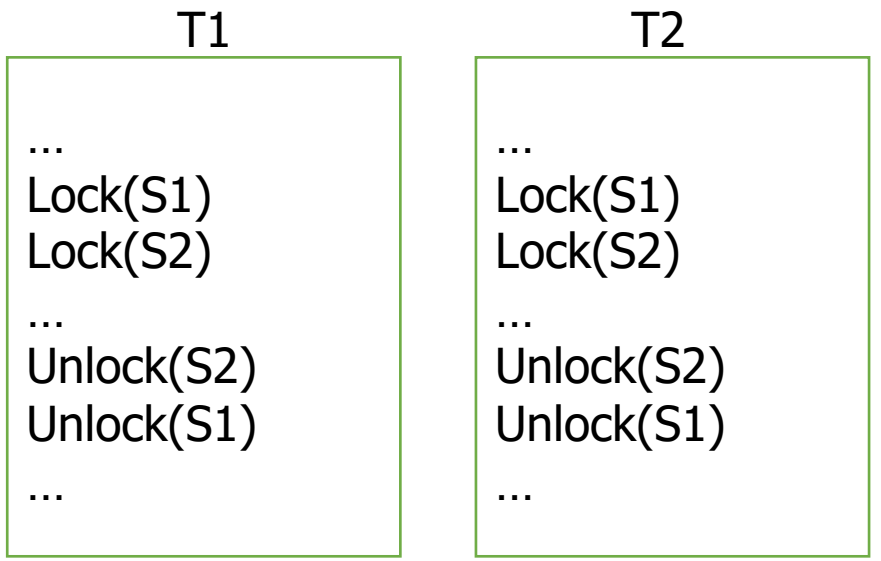


Deadlock Prevention

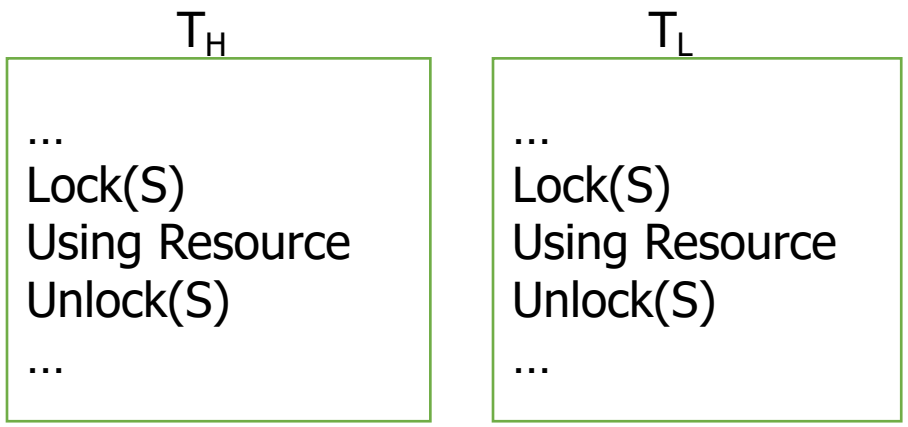
- Prevent circular wait situation
  - Imposes a total ordering of all locks (or resources) in the system and makes each task request locks only in the predefined lock order

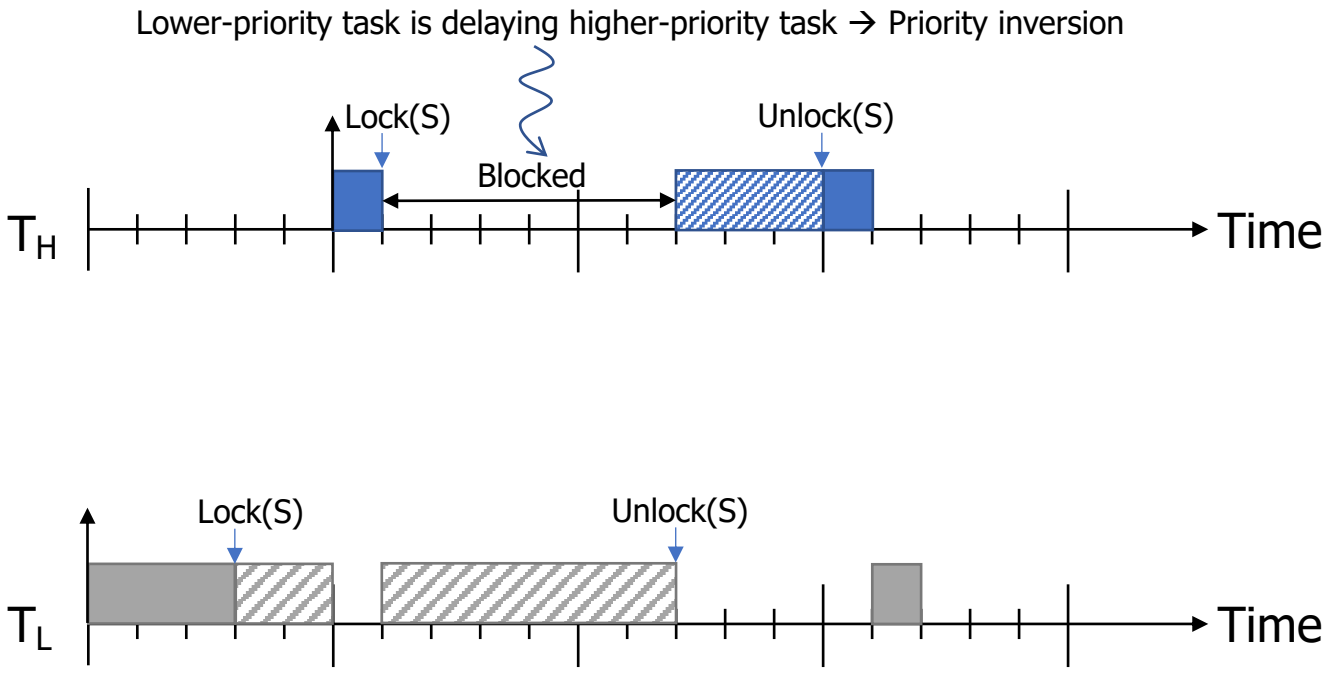
Example Lock Order

S1 → S2 → S3 → …

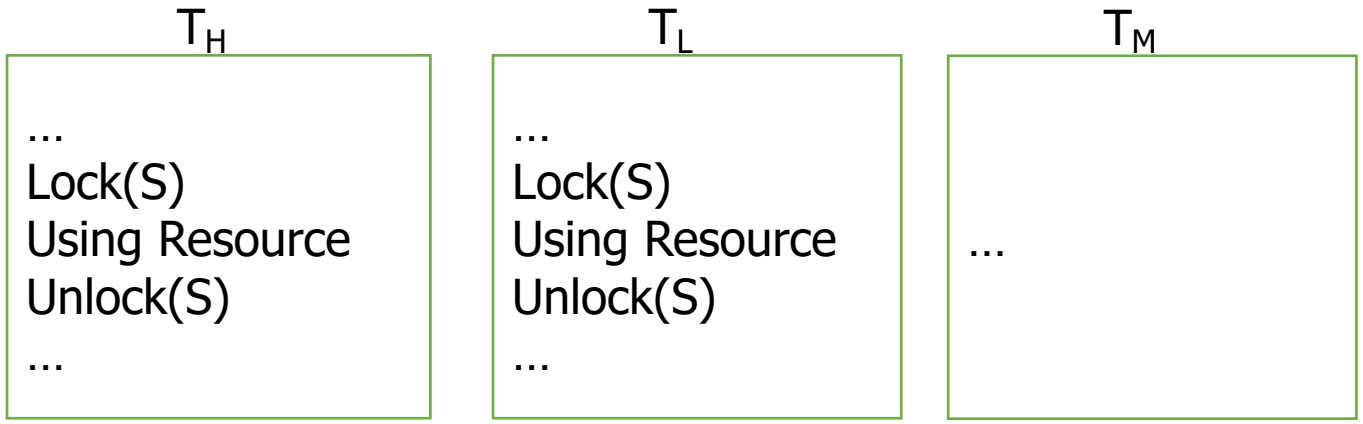


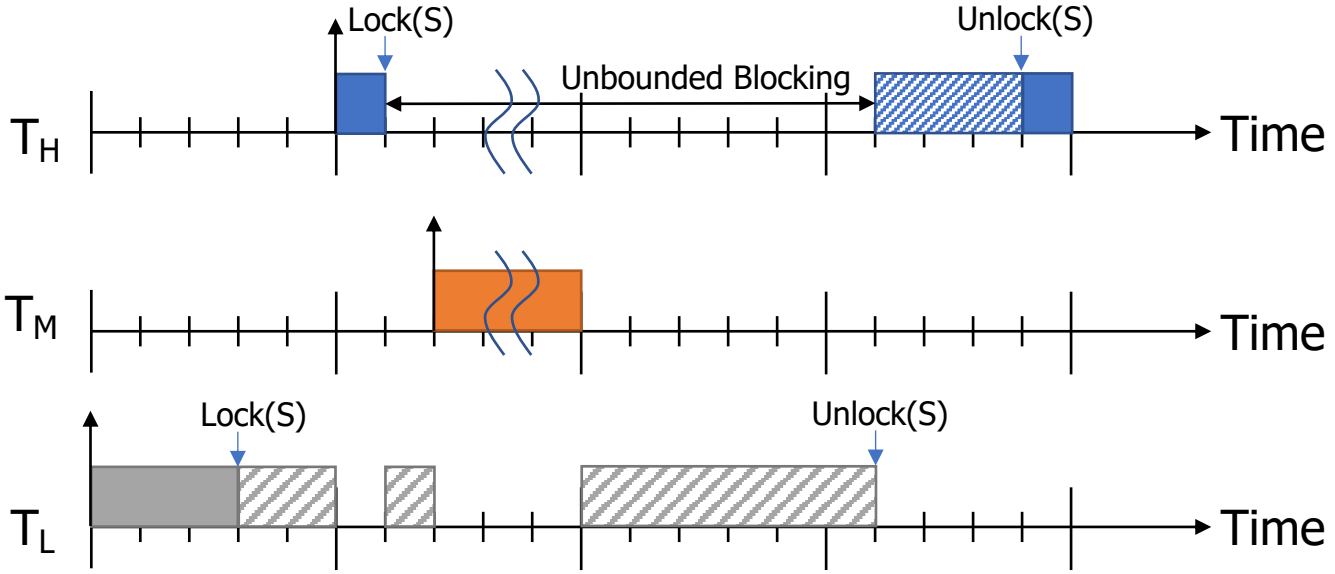
Priority Inversion





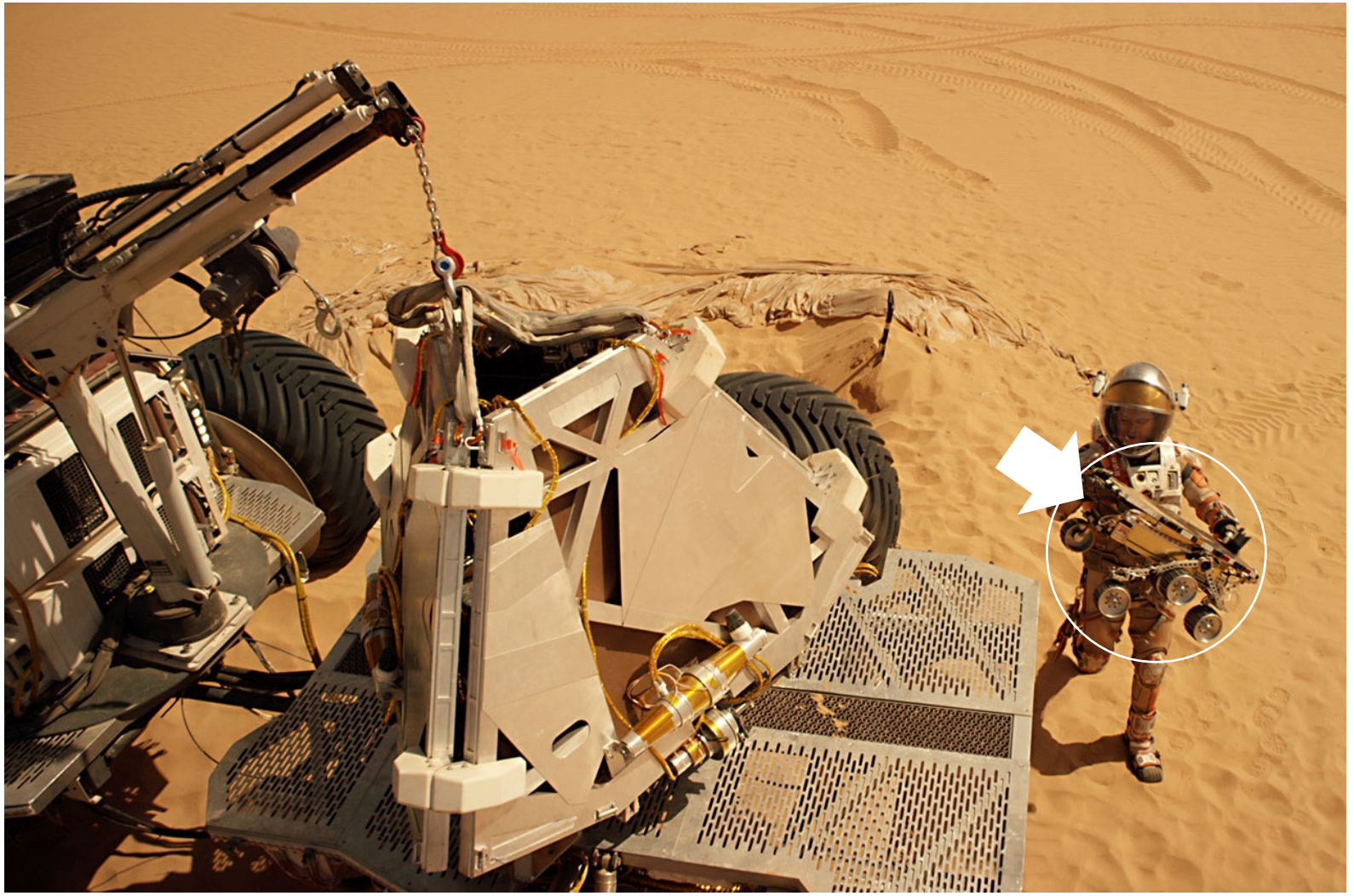
Unbounded Priority Inversion





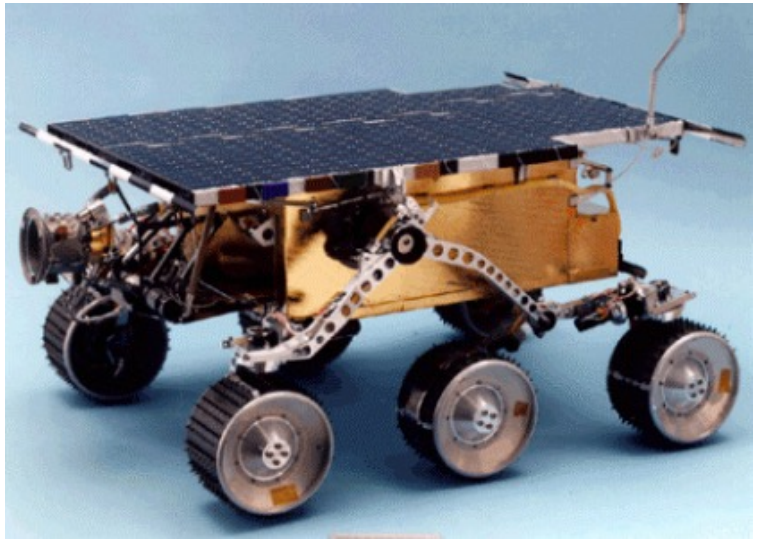
The Martian


A RIDLEY SCOTT FILM
BRING
HIM
HOME
MATT DAMON
THE MARTIAN
HARRY GREGSON-WILLIAMS
SIMON KINBERG RIDLEY SCOTT MICHAEL SCHAEFER ADITYA SOOD MARK HUFFAM
ANDY WEIR
DREW GODDARD
RIDLEY SCOTT
OCTOBER



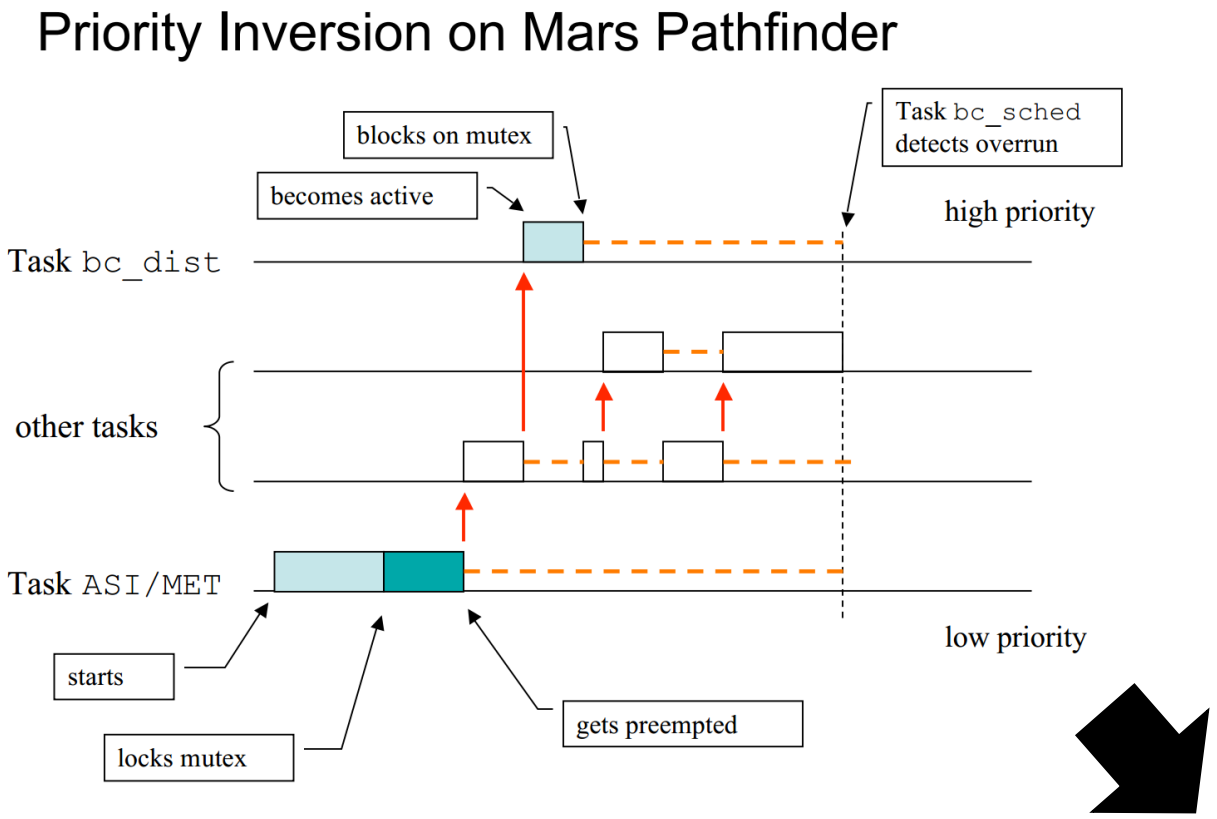
What really happened on Mars?

- Landing on July 4, 1997
- Repeated unexpected system resets
- Deadline miss by priority inversion causing the watchdog task reset the whole system



Contents heavily borrowed from
http://retis.sssup.it/~marko/2009/mars_explorer.pdf and
http://www.cse.nd.edu/~cpoellab/teaching/cse40463/slides6.pdf
http://research.microsoft.com/en-us/um/people/mbj/mars_pathfinder/authoritative_account.html

What really happened on Mars?



Resolution:
NASA patched the lander's software to enable priority inheritance

Questions